Provably Good Scheduling of LLM Agents under KV-Cache Pressure

Stephen Gwon [1] **Eric Sui** [1] **Jonathan Wu** [1]

Abstract

LLM agents alternate between model calls and tool calls of unpredictable length, and many agents share one GPU. When KV-cache memory is scarce, the order in which the engine serves agents decides how long every task takes. Today's engines schedule individual requests first-come-first-served, and agent-aware systems use heuristics without guarantees. We model agent serving as stochastic scheduling of task graphs with random tool delays on a batched engine with bounded KV memory, and derive a *task index* Z' that ranks whole tasks by their cost to the GPU, including the memory they hold and the recomputation that preemption causes. Scheduling by this index is within a factor of 2 of the optimal non-anticipatory policy without memory limits, and within a constant factor with KV memory, given $3\times$ engine speed and extra memory. Used as the request priority in vLLM, the index cuts the **median task completion time by 37%** (42 s to 27 s) and the mean by 19% for multi-hop question-answering agents on A40 GPUs under KV pressure, improves **all seven workloads**, and beats a Continuum-style agent scheduler by 47% on the median.

1. Problem

An LLM agent task is a small program: it calls the model, calls tools (search, code, APIs) whose latency is long and random compared with a model step, often several in parallel (Kim et al., 2024), and calls the model again on the results (Yao et al., 2023). A serving engine such as vLLM (Kwon et al., 2023) sees only the individual model calls and admits them first-come-first-served (FCFS). While an agent waits for a tool, its KV cache either occupies scarce GPU memory or is evicted and later recomputed.

Under memory pressure only a few agents fit on the GPU at once, so the order in which tasks are served dominates latency. Per-request FCFS is a poor order for agents: a task's latency is set by its *last* model call, and a long task that arrives first blocks many short tasks behind it.

Currently, there exists several agent-aware systems that schedule whole programs with heuristics. For example, Continuum (Li et al., 2025) pins an agent's KV cache with a time-to-live during tool calls and serves programs FCFS, Autellix (Luo et al., 2025) ranks programs by least attained service, and LAMPS (Shahout et al., 2024) ranks requests by memory use over time. None has a performance guarantee. On the theory side, online algorithms for KV-constrained LLM scheduling (Jaillet et al., 2025) treat single requests, without tool calls, task graphs or random durations, and classical stochastic scheduling (Möhring et al., 1999; Skutella & Uetz, 2005) has no batching, KV memory or tool delays. To our knowledge, ours is the first scheduling policy for agent serving with a provable approximation guarantee.

There are several challenges we faced during the project. First, tool delays are random and unknown when decisions are made. Second, GPU time and KV memory are coupled: evicting a waiting agent frees memory but costs recomputation, and a batched step slows down as more sequences join it. Third, latency belongs to whole tasks, while the engine schedules individual calls. Lastly, a guarantee must survive vLLM's actual admission, preemption and caching rules.

2. Design

Model. A task J arrives at time r_J with weight w_J and is a directed acyclic graph of LLM calls (nodes); tool calls are random delays on its edges. Node y has a new prompt of q_y tokens, ρ_y tokens of context from earlier calls, and X_y output tokens, so its peak KV footprint is $\ell_y = \rho_y + q_y + X_y$; $L_{\max}$ is the largest footprint. A batched step that decodes B sequences and prefills P tokens takes $a + cB + \alpha P$ seconds, and the KV cache holds M' tokens. For Qwen3-8B on an A40 we measured $a = 27.8\,\text{ms}$, $c = 0.54\,\text{ms}$ and $\alpha = 0.11\,\text{ms}$ per token. The objective is the expected weighted sum of task completion times, $\mathbb{E}[\sum_J w_J C_J]$. The

[1]Harvard University, CS 243 (Fall 2026). Midterm report. Correspondence to: Stephen Gwon <stephengwon@gmail.com>.

Proceedings of the 42nd International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025. Copyright 2025 by the author(s).

Table 1. The index Z' (seconds) and its four terms for each task shape at 16k tokens of KV memory ($L_{\max} = 8{,}184$, $M = 7{,}816$), from the token-count estimates used by the client.

Task	Prefill	Decode	Preempt	Area	Z'
Fan-out, 2	0.26	0.10	2.15	0.63	3.14
Fan-out, 4	0.49	0.10	2.38	1.08	4.05
Fan-out, 8	0.94	0.10	2.83	1.99	5.87
ReAct, 2	0.42	0.09	3.24	0.79	4.54
ReAct, 4	1.32	0.14	6.02	2.28	9.76
ReAct, 6	2.72	0.19	9.30	4.52	16.73

benchmark OPT is the best non-anticipatory policy, which knows the distributions of tool delays but not their outcomes.

The task index. Each task gets one number when it arrives:

$$Z'_J = \sum_{y \in J} \left[\alpha(q_y + \rho_y) + cX_y + \alpha(\ell_y + L_{\max}) + \frac{aX_y\ell_y}{M} \right], \tag{1}$$

where $M = M' - L_{\max}$. The four terms are the node's prefill work (including recomputing context evicted during a tool wait), its decode work, the recomputation its arrival can cause by preempting lower-priority requests, and its memory-time area. Each term bounds one way the task can delay lower-priority tasks (Appendix B). Tasks are served in increasing order of Z'_J/w_J, a memory-aware version of Smith's rule (Smith, 1956). The index is computed once, from token-count estimates, and never changes. Table 1 gives its value for each task shape we evaluate.

Guarantees.

Theorem 1 (no memory limit)**.** *For task graphs with known node sizes and arbitrary random tool delays on m parallel machines, the online preemptive policy that serves tasks in decreasing order of w_J/W_J, where W_J is the total work of J, satisfies $\mathbb{E}[\sum w_J C_J] \le 2\,\mathrm{OPT}$.*

Theorem 2 (batched engine with KV memory)**.** *Suppose token counts are known when a task arrives. Let policy P serve tasks in increasing order of Z'_J/w_J, admit the longest priority prefix of ready requests whose footprints fit in M', preempt the lower-priority requests that no longer fit, and evict KV during tool waits. On the fluid engine at $3\times$ speed, $\mathbb{E}[\sum w_J C_J] \le (1 + \beta + 2\lambda)\,\mathrm{OPT}$, where OPT runs at speed 1 with memory M, $\beta = \max_y \alpha\rho_y/(\alpha q_y + aX_y)$ is the largest share of recomputed context, and $\lambda = \max_J Z'_J/(\sigma_J + A_J/M)$ compares each task's index with its GPU work σ_J and memory-time area A_J.*

Theorem 1 extends Megow and Schulz (Megow & Schulz, 2004) to task graphs with random tool delays. The delays enter only through the critical path, and their lower bound shows that the factor 2 is tight. Theorem 2 is new; its key step charges every preemption to the arrival that caused it.

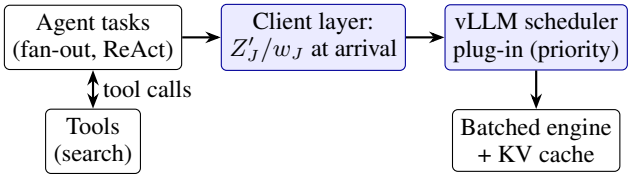


Figure 1. Implementation. The client layer computes each task's index once and attaches it as the priority of every model call of the task; a scheduler plug-in loaded with `-scheduler-cls` applies it inside vLLM. Shaded boxes are ours.

For our workload at 16k tokens of memory the constant is 9.5. This is a worst case: on 200 small instances checked exactly, the policy stays within a factor of 1.45 of the lower bounds used in the proof. Proofs are in Appendices A and B.

Theorem 2 assumes token counts are known, but output lengths are random. For chain tasks we proved two extensions with Gittins-index versions of the policy. If output lengths are revealed token by token and memory never binds, the ratio is $1 + \beta + \lambda'$, where λ' keeps only the prefill and decode terms of Z'. With bounded memory, reserving each node's maximum output length gives a constant that also grows with the variability of output lengths. We also proved a 2-approximation for chain tasks with random sizes, that keeping KV across tool waits preserves Theorem 2 if kept KV is evicted first, and that the best keep-or-evict time-to-live for a known delay distribution is within $e/(e-1)$ of clairvoyant. Two independent verification passes audited the proofs, and every inequality is also checked exactly on small instances, with rational arithmetic and the Z3 solver.

Lower bounds explain the model's choices. A policy that never evicts a waiting agent's KV has no constant ratio, because memory then caps how many agents can wait on tools at once; evicting lets every tool wait overlap. Under adversarial arrivals no online policy has a constant ratio for flow time (Jaillet et al., 2025), so we bound completion time. With random node sizes, any fixed-priority policy can lose a factor $1 + \Delta$ against a preemptive optimum, where Δ bounds the squared coefficient of variation of the sizes; this is why the random-length results use Gittins indices. The $3\times$ speed comes from the analysis, which emulates a separate prefill server and decode machines on one engine; we do not know whether it is necessary.

Implementation (Figure 1). The system needs no fork of vLLM. A client layer (140 lines) computes Z'_J from per-shape token-count estimates when a task arrives and sends it as the priority of each of the task's model calls. A scheduler subclass (360 lines), loaded through vLLM 0.32's `-scheduler-cls` option, implements the rest of policy

Table 2. All tasks at 16k tokens of KV memory, seven seeds (653 paired tasks). "Faster": share of tasks that finish earlier than under vLLM.

Scheduler	Mean (s)	Median (s)	Faster
vLLM (FCFS)	50.3	42.0	–
Continuum-like	52.1	50.7	48%
Autellix-like*	65.9	47.3	49%
Ours (Z' index)	**40.6**	**26.7**	**79%**

*Run with our preemptive admission rule.

P (priority-prefix admission with preemption, one prefill per step) and rules for KV kept during tool waits. Every rule is a switch, so the baselines run through the same code. In this report, *Ours* is the index on vLLM's stock priority scheduler. This is not exactly policy P: stock vLLM never preempts a running request to admit a waiting one, so Theorem 2 does not directly cover it. The full policy P is evaluated as an ablation (Section 4).

3. Initial Evaluation

Our setup used four NVIDIA A40 GPUs (48 GB), one vLLM 0.32 server per GPU, Qwen3-8B (Qwen Team, 2025) in bf16. Agents answer multi-hop HotpotQA questions (Yang et al., 2018) with a search tool over HotpotQA paragraphs, in two shapes: *fan-out* (plan, then 2, 4 or 8 parallel searches, then answer) and *ReAct* (2, 4 or 6 search-and-reason turns). Search results are up to 1,000 tokens; tool delays are uniform on 0.5–4 s with probability 0.7 and on 4–12 s otherwise. Tasks arrive as a Poisson process at 1 per second for 90 s (about 93 tasks per run). To emulate the memory pressure of a busy multi-tenant server, we cap the KV cache at 16k tokens (full capacity: 167k). Each *seed* fixes the arrival times, task mix and tool delays, and all schedulers see the same seeds, so comparisons are paired task by task. We report task completion time, from arrival to final answer.

Baselines. *vLLM*: stock scheduler, requests FCFS. *Continuum-like*: our reimplementation of Continuum (Li et al., 2025): KV pinned during tool calls with a time-to-live, programs served FCFS, pinned programs first. *Autellix-like*: program-level least-attained-service ranking (Luo et al., 2025).

Results. Figure 2 and Table 2 show the main result. Under memory pressure, the index cuts the median completion time from 42 s to 27 s (-37%, 95% CI 31–43%) and the mean from 50.3 s to 40.6 s (-19%, CI 15–23%); 79% of tasks finish earlier, and every workload improves (Figure 2b). Against the Continuum-like scheduler, the median drops by 47% (CI 36–54%) and the mean by 22%. The confidence intervals come from a two-level bootstrap that resamples workloads and then tasks, so they include run-to-run variation. Fan-out tasks gain the most: their mean falls by 34% (32.9 s to 21.6 s), half of the Continuum-like scheduler's 42.8 s.

When does it help? Figure 3 repeats the experiment with more KV memory (three seeds each at 24k, 30k and full memory). The median falls by 22% at 24k and 13% at 30k, and at full memory the index matches vLLM. As the model predicts, the index matters when memory limits how many agents run at once: busy multi-tenant servers, long contexts and large tool outputs.

4. Remaining Challenges and Plan

1. Isolate what the index contributes. With equal weights, Z' ranks tasks by estimated cost. At 16k tokens the preemption term $\alpha(\ell_y + L_{\max})$ makes up 48–71% of Z' (Table 1), although stock admission never preempts for a waiting request. We will compare Z' with simpler rankings on the same stock admission: smallest estimated task first, Z' without the preemption term, Autellix's ranking, and a LAMPS-style memory-area ranking (Shahout et al., 2024). This decides whether the memory and recomputation terms of Eq. (1) matter in practice.

2. Protect the longest tasks with aging. Ranking by cost makes the largest tasks wait: six-turn ReAct agents (17% of tasks) finish 14% later than under vLLM, and their 90th percentile does not improve (106 s versus 101 s). We will give task J the priority $r_J + \gamma Z'_J/w_J$ (smaller is served first), which interpolates between task-level FCFS ($\gamma = 0$) and the index ($\gamma \to \infty$). A later task K is served before J only if it arrives less than $\gamma(Z'_J/w_J - Z'_K/w_K) \le \gamma Z'_J/w_J$ seconds after J, so no task waits behind an endless stream of smaller ones. At 16k memory Z' ranges from 3.1 s (two-search fan-out) to 16.7 s (six-turn ReAct), so with $\gamma \le 8$ a long task can be overtaken only by tasks arriving within about 2 minutes of it. The rule is implemented in the client and unit-tested. The priority is still fixed at arrival, but it no longer follows Smith's rule, so Theorem 2 does not yet cover it; we will extend the proof using the overtaking bound.

3. Make preemption and KV retention cheap. Theorem 2 needs admission in priority order with preemption. In vLLM a preempted request loses its KV and recomputes its whole context, so in our ablations the preemptive admission of policy P was slower than the index on stock admission. Keeping KV across tool waits also gave no measurable gain at this scale. We plan three engine changes. (a) Preempt and evict by swapping KV to CPU memory through vLLM's KV-offload connector, so that resuming costs a transfer instead of a recomputation; the model treats a swap as a fixed delay, like a tool call. (b) Evict kept KV blocks in order of recomputation saved per unit of memory

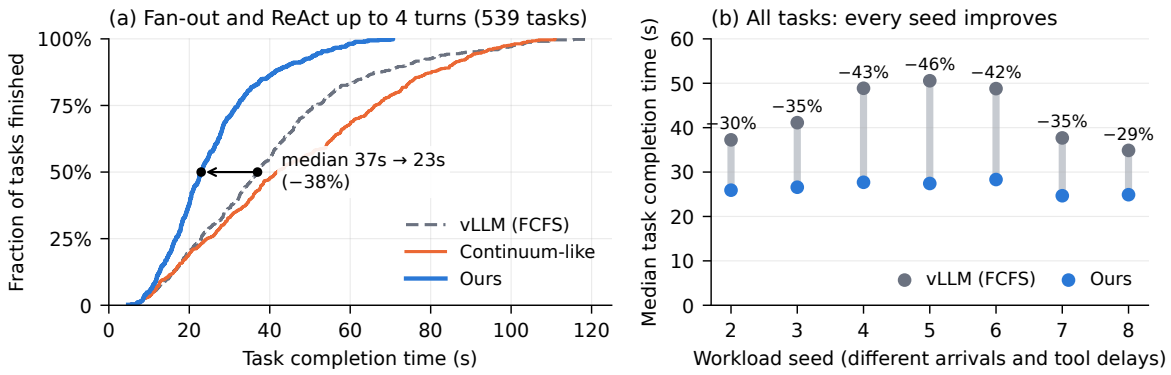


Figure 2. Task completion times at 16k tokens of KV memory, seven workloads. (a) Fan-out tasks and ReAct tasks with up to four turns (539 of 653 tasks): the index finishes 90% of them within 45 s versus 74 s with vLLM, and the slowest within 71 s versus 120 s. Six-turn ReAct tasks are discussed in Section 4. (b) Median over all tasks, per workload: the index improves every workload, by 29–46% (mean: 11–24%).

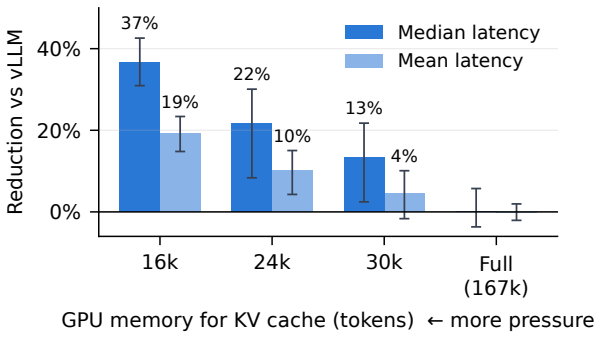


Figure 3. Latency reduction of our index over vLLM as KV memory grows (95% CIs). The tighter the memory, the larger the gain; with ample memory the order barely matters and the index costs nothing.

and expected remaining wait instead of least-recently-used; the guarantee allows any eviction order among kept blocks. (c) Price memory online from the free-memory fraction, so agents keep KV across tool waits only when memory is plentiful. On the theory side, we will look for an admission rule without preemption that keeps a constant factor.

4. Broaden the evaluation. More arrival rates and memory sizes, a larger model, longer contexts and tool outputs (where keeping KV should pay off), and the ratio of each run to the lower bound on OPT, which tests how pessimistic the proven constant is.

Timeline. Items 1 and 2 by the presentation (12/1); items 3 and 4 and the final report by 12/16. Our earlier study of KV preloading of tool results is complete and serves as a fallback.

References

Jaillet, P., Jiang, J., Mellou, K., Molinaro, M., Podimata, C., and Zhou, Z. Online scheduling for LLM inference with KV cache constraints. *arXiv preprint arXiv:2502.07115*, 2025.

Kim, S., Moon, S., Tabrizi, R., Lee, N., Mahoney, M. W., Keutzer, K., and Gholami, A. An LLM compiler for parallel function calling. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

Li, H., Mang, Q., He, R., Zhang, Q., Mao, H., Chen, X., Cheung, A., Gonzalez, J., and Stoica, I. Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live. *arXiv preprint arXiv:2511.02230*, 2025.

Luo, M., Shi, X., Cai, C., Zhang, T., Wong, J., Wang, Y., Wang, C., Huang, Y., Chen, Z., Gonzalez, J. E., and Stoica, I. Autellix: An efficient serving engine for LLM agents as general programs. *arXiv preprint arXiv:2502.13965*, 2025.

Megow, N. and Schulz, A. S. On-line scheduling to minimize average completion time revisited. *Operations Research Letters*, 32(5):485–490, 2004.

Möhring, R. H., Schulz, A. S., and Uetz, M. Approximation in stochastic scheduling: The power of LP-based priority policies. *Journal of the ACM*, 46(6):924–942, 1999.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Shahout, R., Liang, C., Xin, S., Lao, Q., Cui, Y., Yu, M., and Mitzenmacher, M. Fast inference for augmented large language models. *arXiv preprint arXiv:2410.18248*, 2024.

Skutella, M. and Uetz, M. Stochastic machine scheduling with precedence constraints. *SIAM Journal on Computing*, 34(4):788–802, 2005.

Smith, W. E. Various optimizers for single-stage production. *Naval Research Logistics Quarterly*, 3(1–2):59–66, 1956.

Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

A. Proof of Theorem 1

Setting. There are m identical machines. Node x of task J has a known size p_x, and an edge $u \to v$ carries a random delay $D_{uv} \geq 0$, with any joint law independent of the schedule. Node v becomes *available* at time a_v, the first time after r_J at which every predecessor u has completed and D_{uv} has elapsed. Let $W_J = \sum_{x \in J} p_x$, and let CP_J be the longest path of J counting node sizes and delays. The policy ranks tasks by w_J/W_J at arrival (ties by arrival) and ranks the nodes of a task in topological order. $K \preceq J$ means that K is ranked no lower than J, including $K = J$. At every instant the policy runs the m highest-ranked available nodes, preempting as needed.

Lemma 1. *On every realization of the delays, $C_J \leq r_J + CP_J + \frac{1}{m}\sum_{K \preceq J} W_K$ for every task J.*

Proof. Let v_k be the last node of J to complete. Its availability time a_{v_k} is either r_J or $C_u + D_{uv_k}$ for some predecessor u; take that u as v_{k-1} and repeat until reaching a node v_1 with $a_{v_1} = r_J$. This splits $[r_J, C_J]$ into delay intervals $[C_{v_{i-1}}, a_{v_i}]$ and node intervals $[a_{v_i}, C_{v_i}]$. During a node interval, v_i is available, so at each instant either v_i runs or all m machines run nodes ranked above it, which belong to tasks $K \preceq J$. The first kind of instant totals at most p_{v_i}. The second kind, summed over all node intervals, totals at most $\frac{1}{m}\sum_{K \preceq J} W_K$, because the machines can do at most that much work for these tasks. The delays and node sizes along the path sum to at most CP_J. □

Proof of Theorem 1. Two lower bounds hold on every realization, even for a clairvoyant schedule that knows the delays in advance, and hence for OPT. First, $C_J \geq r_J + CP_J$ in any schedule, so $\mathrm{OPT} \geq LB_{cp} := \sum_J w_J(r_J + \mathbb{E}[CP_J])$. Second, any m-machine schedule can be simulated on one machine of speed m that ignores release dates and delays, with no task finishing later. On that machine Smith's rule is optimal, so $\mathrm{OPT} \geq \frac{1}{m}\mathrm{Smith}(W)$, where $\mathrm{Smith}(W) = \sum_J w_J \sum_{K \preceq J} W_K$ with tasks ordered by decreasing w_K/W_K. This is exactly the policy's order, so multiplying Lemma 1 by w_J, summing over J and taking expectations gives $\mathbb{E}[\sum_J w_J C_J] \leq LB_{cp} + \frac{1}{m}\mathrm{Smith}(W) \leq 2\,\mathrm{OPT}$. For single-node tasks without delays the policy is preemptive WSPT, for which Megow and Schulz (Megow & Schulz, 2004) show that the factor 2 is tight. □

B. Proof of Theorem 2

Assumptions. The token counts ρ_y, q_y, X_y of every node of J are known when J arrives. Tool delays are random, with any law independent of the schedule. OPT does not prefill a node's prompt before the node is available (see the remarks at the end).

Analysis model. We analyze P in a hybrid model with one prefill server and $m = a/c$ decode machines. The server prefills $1/\alpha$ tokens per second, one node at a time, preemptively; a paused prefill keeps its KV. Each decode machine decodes $1/a$ tokens per second. KV memory is M'. The fluid engine at speed s gives prefill a share u of its time, so prefill runs at su/α tokens per second and each of B decoding sequences at $s(1-u)/(a+cB)$. With $s = 3$ and $u = 1/3$, prefill runs at $1/\alpha$ and each sequence at $2/(a+cB) \geq 1/a$ whenever $B \leq m$, so the speed-3 engine can reproduce any hybrid schedule exactly; no speed below 3 suffices. It is therefore enough to bound P in the hybrid model against OPT on the engine at speed 1.

Policy P. Tasks are ranked by increasing Z'_J/w_J at arrival, and nodes become available as in Appendix A. The *admitted* nodes are the longest prefix, in priority order, of the available unfinished nodes whose footprints ℓ_y sum to at most M'; the prefix stops at the first node that does not fit. When a newly available node enters the prefix, the admitted nodes that no longer fit are *kicked*: their KV is discarded, and on readmission they recompute their prompt and generated tokens as prefill, then continue decoding where they stopped. The server prefills the highest-ranked admitted node in its prefill phase, and the m highest-ranked admitted nodes in their decode phase decode. A finished node's KV is freed, so its children recompute their ρ context tokens.

Lemma 2 (kick charge)**.** *When node y becomes available, the nodes kicked at that instant have total footprint less than $\ell_y + L_{\max}$, so they cause less than $\kappa_y := \alpha(\ell_y + L_{\max})$ seconds of recomputation.*

Proof. Before the event, the admitted nodes have total footprint $S_{\text{old}} \le M'$; let S_{above} be the part ranked above y. If $S_{\text{above}} + \ell_y > M'$, the new prefix stops at y, and the kicked nodes are those ranked below y, of total $S_{\text{old}} - S_{\text{above}} \le M' - S_{\text{above}} < \ell_y$. Otherwise y is admitted. If the new prefix stops at a node of footprint $\ell_{\text{next}} \le L_{\max}$, its total satisfies $S_{\text{new}} > M' - \ell_{\text{next}}$; it consists of y and the old nodes that stay admitted, so the kicked total is $S_{\text{old}} - (S_{\text{new}} - \ell_y) < \ell_y + \ell_{\text{next}} \le \ell_y + L_{\max}$. If the prefix does not stop, nothing is kicked. A kicked node recomputes at most its footprint. Completions only free memory and extend the prefix, so they kick nothing. Simultaneous arrivals are handled the same way, with their total footprint in place of ℓ_y. □

Lemma 3 (pathwise bound)**.** *On every realization of the delays, $C_J \le r_J + CP'_J + \sum_{K \preceq J} Z'_K$, where CP'_J is the longest path of J with node lengths $\alpha(q_y + \rho_y) + aX_y$ plus delays.*

Proof. Build the path $v_1, \dots, v_k$ as in Lemma 1, and fix a node v on it. Every instant of $[a_v, C_v]$ falls into one of four cases.

(P) v is prefilled or decoded. This totals at most $\alpha(q_v + \rho_v) + aX_v$; recomputation of v after a kick is charged to the kicker in (C).

(C) The server prefills a node ranked above v, including recomputation after kicks. A kicked node is ranked below its kicker, so all of this work belongs to nodes and kickers of tasks $K \preceq J$. By Lemma 2, these instants total at most $\sum_{K \preceq J} \sum_{y \in K} [\alpha(q_y + \rho_y) + \kappa_y]$.

(A) All m decode machines decode nodes ranked above v. Kicked nodes resume where they stopped, so each token is decoded once, at a total rate of m/a tokens per second. These instants total at most $\sum_{K \preceq J} \sum_{y \in K} (a/m) X_y = \sum_{K \preceq J} \sum_{y \in K} cX_y$.

(B) v is not admitted, no admitted node is in its prefill phase, and fewer than m admitted nodes exist. Then every admitted node is decoding. The prefix stopped at a node ranked no lower than v, so the admitted nodes are all ranked above v and their footprints exceed $M' - L_{\max} = M$. A node y decodes for aX_y seconds in total while holding ℓ_y tokens, so these instants total at most $\sum_{K \preceq J} \sum_{y \in K} aX_y \ell_y / M$.

The cases are exhaustive. If v is admitted in its prefill phase but not served, the server prefills a node ranked above it (C). If v is admitted in its decode phase but not decoding, m higher-ranked nodes decode (A). If v is not admitted, the admitted nodes are ranked above v, and the instant is (C), (A) or (B). Summing over the path, the (P) instants and the delays total at most CP'_J, and (A), (B) and (C) total at most $\sum_{K \preceq J} Z'_K$ by Eq. (1). □

Lower bounds on OPT**.** OPT runs the engine at speed 1 with memory M. Let $\sigma_y = \alpha q_y + cX_y$ and $A_y = (\rho_y + q_y)(a + c)X_y$, and let σ_J and A_J be their sums over J's nodes. Both bounds below hold on every realization.

(i) *Critical path.* Once node y is available, OPT needs at least $\alpha q_y + (a + c)X_y$ seconds to finish it: its q_y new tokens must be prefilled before its first decode step, and each of its X_y decode steps lasts at least $a + c$. (The parent's KV may be kept, so ρ_y is free.) Hence $C_J \ge r_J + CP^*_J$, where CP^*_J uses these node lengths, and $\text{OPT} \ge LB_{cp} := \sum_J w_J (r_J + \mathbb{E}[CP^*_J])$.

(ii) *GPU time and memory area.* A step lasts at least c per decoded sequence plus α per prefilled token, so the GPU work σ done by time t totals at most t. During each of its decode steps, node y holds at least $\rho_y + q_y$ tokens for at least $a + c$ seconds, and memory holds at most M tokens, so the area A accumulated by time t is at most Mt. Ordering tasks by their completion times under OPT, $\sum_{K \text{ up to } J} (\sigma_K + A_K/M) \le 2C_J$, hence $\text{OPT} \ge \frac{1}{2}\text{Smith}(\sigma + A/M)$.

Proof of Theorem 2. Each node satisfies $\alpha(q_y + \rho_y) + aX_y \le (1+\beta)(\alpha q_y + aX_y) \le (1+\beta)(\alpha q_y + (a+c)X_y)$, so $r_J + CP'_J \le (1+\beta)(r_J + CP^*_J)$ on every realization. By the definition of λ, $Z'_K \le \lambda(\sigma_K + A_K/M)$. P orders tasks by Smith's rule on Z', so $\sum_J w_J \sum_{K \preceq J} Z'_K = \text{Smith}(Z') \le \lambda\,\text{Smith}(\sigma + A/M)$. Multiplying Lemma 3 by w_J, summing and taking expectations gives $\mathbb{E}[\sum_J w_J C_J] \le (1 + \beta) LB_{cp} + \lambda\,\text{Smith}(\sigma + A/M) \le (1 + \beta + 2\lambda)\text{OPT}$. □

Remarks. (1) If OPT may prefill tool results that return early while other tool calls of the same node are still running, bound (i) fails. Setting $q_y = 0$ in (i) for such nodes restores it, at a slightly larger constant (9.6 instead of 9.5 for our workload). (2) For our workload at 16k tokens, $\beta = 0.36$ and $\lambda = 4.05$. The preemption charge κ_y dominates λ, because outputs are short. (3) The proof is for the fluid engine. For discrete steps we have only a proof sketch, which adds one step time per node and per completed prefill.